

CS3014 - Estructuras de Datos Avanzadas

Fractional Cascading I

Semana 5

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

8 de septiembre de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



- ▶ Repaso: Range Tree 2D, y su costo escondido

¿Qué aprenderemos hoy?



- ▶ Repaso: Range Tree 2D, y su costo escondido
- ▶ *Fractional Cascading*: la técnica general

¿Qué aprenderemos hoy?



- ▶ Repaso: Range Tree 2D, y su costo escondido
- ▶ *Fractional Cascading*: la técnica general
- ▶ *Layered Range Trees*: aplicándola al árbol

¿Qué aprenderemos hoy?



- ▶ Repaso: Range Tree 2D, y su costo escondido
- ▶ *Fractional Cascading*: la técnica general
- ▶ *Layered Range Trees*: aplicándola al árbol
- ▶ Range Trees en 3D, con *fractional cascading*

Repaso

Repaso: Range Tree 2D



De la clase pasada

Un BST **primario** en x ; cada nodo guarda, además, una estructura **secundaria** (otro BST) con los y de su subárbol. La consulta $[x_1, x_2] \times [y_1, y_2]$ halla los $O(\lg n)$ nodos canónicos de x , y busca $[y_1, y_2]$ en **cada** una de sus estructuras secundarias.

Repaso: Range Tree 2D



De la clase pasada

Un BST **primario** en x ; cada nodo guarda, además, una estructura **secundaria** (otro BST) con los y de su subárbol. La consulta $[x_1, x_2] \times [y_1, y_2]$ halla los $O(\lg n)$ nodos canónicos de x , y busca $[y_1, y_2]$ en **cada** una de sus estructuras secundarias.

El costo

$O(\lg n)$ nodos canónicos $\times O(\lg n)$ por búsqueda secundaria = $O(\lg^2 n)$ por consulta (más $O(k)$ para reportar).

¿De dónde sale el desperdicio?



Observación clave

En los $O(\lg n)$ nodos canónicos, estamos buscando el **mismo** valor y_1 (y el mismo y_2), una y otra vez, en $O(\lg n)$ estructuras **distintas** pero **relacionadas**.

¿De dónde sale el desperdicio?



Observación clave

En los $O(\lg n)$ nodos canónicos, estamos buscando el **mismo** valor y_1 (y el mismo y_2), una y otra vez, en $O(\lg n)$ estructuras **distintas** pero **relacionadas**.

¿Te suena de algo “buscar el mismo valor en varias listas a la vez”? Es exactamente el problema que vamos a resolver ahora — de forma completamente general primero, y luego aplicándolo a este árbol.

Fractional Cascading

El problema general



Consultas en múltiples listas

Dadas k listas ordenadas $L_1, \dots, L_k$ (cada una de tamaño $\leq n$), buscar un mismo valor x en **todas**.

El problema general



Consultas en múltiples listas

Dadas k listas ordenadas $L_1, \dots, L_k$ (cada una de tamaño $\leq n$), buscar un mismo valor x en **todas**.

¡Piénsalo!

Buscando x en cada lista por separado con búsqueda binaria, ¿cuánto cuesta en total?

El problema general



Consultas en múltiples listas

Dadas k listas ordenadas $L_1, \dots, L_k$ (cada una de tamaño $\leq n$), buscar un mismo valor x en **todas**.

¡Piénsalo!

Buscando x en cada lista por separado con búsqueda binaria, ¿cuánto cuesta en total?

Lo obvio: $O(k \lg n)$

Queremos hacerlo mejor: $O(k + \lg n)$ — **una sola** búsqueda binaria cara, y $O(1)$ por cada lista adicional.

La idea: promover elementos hacia arriba



Construcción (de abajo hacia arriba)

Sea $L'_k = L_k$. Para i de $k-1$ a 1:

$$L'_i = L_i \cup \{\text{cada elemento par de } L'_{i+1}\}.$$

Cada elemento **promovido** (tomado de L'_{i+1}) guarda un **punteo**: un puntero a su posición exacta en L'_{i+1} .

La idea: promover elementos hacia arriba



Construcción (de abajo hacia arriba)

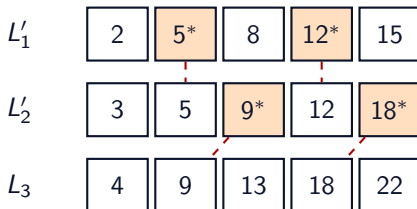
Sea $L'_k = L_k$. Para i de $k-1$ a 1:

$$L'_i = L_i \cup \{\text{cada elemento par de } L'_{i+1}\}.$$

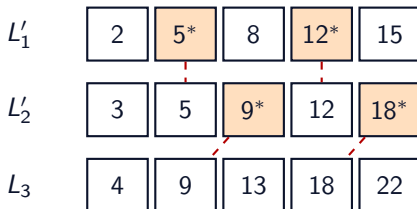
Cada elemento **promovido** (tomado de L'_{i+1}) guarda un **punteo**: un puntero a su posición exacta en L'_{i+1} .

L'_1 contiene elementos de **todas** las listas originales, cada uno con una cadena de puentes hacia abajo.

Visualizando la construcción



Visualizando la construcción



Los elementos con * fueron **promovidos** desde la lista de abajo, y guardan un puente (línea punteada) a su posición original. L_3 no cambia (es la última).

Usando los puentes en una consulta



Buscar x en todas las listas

Usando los puentes en una consulta



Buscar x en todas las listas

1. Una **única** búsqueda binaria de x en L'_1 : $O(\lg n)$.

Usando los puentes en una consulta



Buscar x en todas las listas

1. Una **única** búsqueda binaria de x en L'_1 : $O(\lg n)$.
2. Desde esa posición, seguir el puente más cercano hacia L'_2 : cae a lo más ± 1 posición de donde x realmente iría — ajustar con $O(1)$ comparaciones.

Usando los puentes en una consulta



Buscar x en todas las listas

1. Una **única** búsqueda binaria de x en L'_1 : $O(\lg n)$.
2. Desde esa posición, seguir el puente más cercano hacia L'_2 : cae a lo más ± 1 posición de donde x realmente iría — ajustar con $O(1)$ comparaciones.
3. Repetir el paso anterior para bajar a $L'_3, \dots, L_k$: $O(1)$ cada uno.

Usando los puentes en una consulta



Buscar x en todas las listas

1. Una **única** búsqueda binaria de x en L'_1 : $O(\lg n)$.
2. Desde esa posición, seguir el puente más cercano hacia L'_2 : cae a lo más ± 1 posición de donde x realmente iría — ajustar con $O(1)$ comparaciones.
3. Repetir el paso anterior para bajar a $L'_3, \dots, L_k$: $O(1)$ cada uno.

Conclusión

Total: $O(\lg n)$ (una búsqueda real) + $O(k)$ (bajar puentes) = $O(k + \lg n)$.

¡Piénsalo! ¿Por qué exactamente la mitad?



La pregunta

Promovimos **cada elemento par** (la mitad) de L'_{i+1} hacia L'_i . ¿Qué pasaría si promoviéramos **todos**, en vez de la mitad?

¡Piénsalo! ¿Por qué exactamente la mitad?



La pregunta

Promovimos **cada elemento par** (la mitad) de L'_{i+1} hacia L'_i . ¿Qué pasaría si promoviéramos **todos**, en vez de la mitad?

El tamaño se dispararía

Si $|L'_i| = |L_i| + |L'_{i+1}|$ (todos), el tamaño de L'_1 terminaría siendo $\Theta(kn)$ — de vuelta al costo original. Promoviendo solo la **mitad**: $|L'_i| \leq |L_i| + \frac{1}{2}|L'_{i+1}|$, una serie geométrica que suma $|L'_1| = O(n)$ en total, sin importar k .

Layered Range Trees

Aplicando la idea a nuestro árbol



La diferencia clave

En el árbol de rango, las estructuras secundarias **no** son listas independientes: la de un nodo padre contiene exactamente la **unión** de las de sus dos hijos. Eso es incluso **mejor** que el caso general.

Aplicando la idea a nuestro árbol



La diferencia clave

En el árbol de rango, las estructuras secundarias **no** son listas independientes: la de un nodo padre contiene exactamente la **unión** de las de sus dos hijos. Eso es incluso **mejor** que el caso general.

Reemplazamos cada BST secundario por un arreglo ordenado

Guardamos, en cada nodo, un **arreglo** con los y de su subárbol (no un árbol) — y lo construimos mezclando los arreglos de sus dos hijos, como en Merge Sort.

Construyendo con puentes, de abajo hacia arriba



Al mezclar los arreglos de los dos hijos

Cada elemento del arreglo del **padre** sabe, en $O(1)$, en qué posición cae dentro del arreglo de **cada uno** de sus dos hijos — son exactamente los índices pL y pR que ya usa el propio proceso de MERGE para decidir de dónde tomó cada elemento.

Construyendo con puentes, de abajo hacia arriba

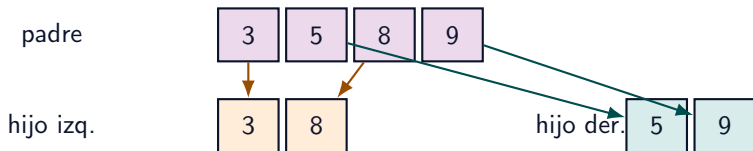


Al mezclar los arreglos de los dos hijos

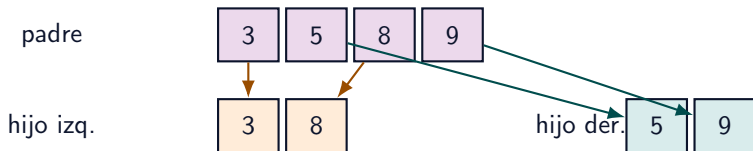
Cada elemento del arreglo del **padre** sabe, en $O(1)$, en qué posición cae dentro del arreglo de **cada uno** de sus dos hijos — son exactamente los índices pL y pR que ya usa el propio proceso de MERGE para decidir de dónde tomó cada elemento.

No hace falta “promover la mitad” como antes: aquí **cada** elemento del padre viene de un hijo, así que **todos** llevan puente — es incluso más simple que el caso general.

Visualizando los puentes padre-hijos



Visualizando los puentes padre-hijos



Cada elemento del arreglo del padre apunta a su posición exacta en **ambos** arreglos hijos — se calcula gratis durante la mezcla, sin ningún trabajo extra.

La consulta: una sola búsqueda binaria, en total



El truco

Al buscar los nodos canónicos de $[x_1, x_2]$, descendemos por el árbol primario **de todos modos** (para eso sirve la búsqueda de 1D). En el nodo **raíz**, hacemos **una** búsqueda binaria real de y_1 en su arreglo: $O(\lg n)$.

La consulta: una sola búsqueda binaria, en total



El truco

Al buscar los nodos canónicos de $[x_1, x_2]$, descendemos por el árbol primario **de todos modos** (para eso sirve la búsqueda de 1D). En el nodo **raíz**, hacemos **una** búsqueda binaria real de y_1 en su arreglo: $O(\lg n)$.

Bajando

En cada paso hacia un hijo, usamos el puntero ya calculado para saltar, en $O(1)$, a la posición de y_1 en el arreglo de **ese** hijo — sin ninguna búsqueda binaria nueva.

La consulta: una sola búsqueda binaria, en total



El truco

Al buscar los nodos canónicos de $[x_1, x_2]$, descendemos por el árbol primario **de todos modos** (para eso sirve la búsqueda de 1D). En el nodo **raíz**, hacemos **una** búsqueda binaria real de y_1 en su arreglo: $O(\lg n)$.

Bajando

En cada paso hacia un hijo, usamos el puntero ya calculado para saltar, en $O(1)$, a la posición de y_1 en el arreglo de **ese** hijo — sin ninguna búsqueda binaria nueva.

Resultado

$O(\lg n)$ nodos canónicos $\times O(1)$ cada uno (tras la primera búsqueda)
= $O(\lg n)$ en total — **un factor de $\lg n$ menos** que la versión ingenua.

Range Tree 2D: antes y después



Ingenuo	<i>Layered</i>
---------	----------------

Range Tree 2D: antes y después



	Ingenuo	<i>Layered</i>
Consulta (contar / re- portar)	$O(\lg^2 n)$	$O(\lg n)$

Range Tree 2D: antes y después



	Ingenuo	<i>Layered</i>
Consulta (contar / re- portar)	$O(\lg^2 n)$	$O(\lg n)$
Espacio	$O(n \lg n)$	$O(n \lg n)$

Range Tree 2D: antes y después



	Ingenuo	<i>Layered</i>
Consulta (contar / re- portar)	$O(\lg^2 n)$	$O(\lg n)$
Espacio	$O(n \lg n)$	$O(n \lg n)$

Mismo espacio, un factor de $\lg n$ menos en el tiempo de consulta — *fractional cascading* no cuesta nada extra de memoria significativo aquí, solo un puntero por elemento.

El resultado, y su generalización



Patrón general para d dimensiones

Sin *cascading*: $O(\lg^{d-1} n)$ (la última dimensión usa un arreglo + búsqueda binaria, ya sin BST completo). Con *cascading* en las dimensiones intermedias, para $d > 2$: $O(\lg^{d-2} n)$.

El resultado, y su generalización



Patrón general para d dimensiones

Sin *cascading*: $O(\lg^{d-1} n)$ (la última dimensión usa un arreglo + búsqueda binaria, ya sin BST completo). Con *cascading* en las dimensiones intermedias, para $d > 2$: $O(\lg^{d-2} n)$.

Cada *cascading* adicional que se pueda encadenar elimina, literalmente, un factor de $\lg n$ del tiempo de consulta — al costo de un puntero extra por elemento, no de una estructura nueva.

Resumen de la sesión



Resumen de la sesión



- ▶ Un **Range Tree 2D** ingenuo (BST de BSTs) cuesta $O(\lg^2 n)$ por consulta — el desperdicio viene de repetir, en $O(\lg n)$ estructuras relacionadas, la búsqueda del **mismo** valor.
- ▶ **Fractional cascading** resuelve el problema general de buscar x en k listas relacionadas en $O(k + \lg n)$: promoviendo la **mitad** de cada lista hacia la anterior, con puentes hacia atrás.
- ▶ Un **Layered Range Tree** aplica exactamente esa idea a las estructuras secundarias del árbol (que ya están relacionadas por mezcla padre-hijos): $O(\lg n)$ por consulta 2D, el mismo espacio de antes.

Gracias

Referencias



Bibliografía citada

- ▶ Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. <https://courses.csail.mit.edu/6.851/spring21/>.
- ▶ de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications*. 3rd ed. Springer.

Agradecimientos



Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.

Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.